

Karakterek és karakterláncok

A mai óra technikai témái:

- `char` és `string`
- karakterkódok és konverzió
- `string` indexelés és `Length`
- `ToString()` és `ToCharArray()`
- karaktervizsgáló műveletek
- `string` keresési és módosító metódusok
- `Join` , `Split` , `Substring`
- `string` interpoláció

1. `char` — egyetlen karakter

A `char` típus egyetlen Unicode UTF-16 karaktert tárol.

```
char letter = 'A';  
char digit = '7';  
char newline = '\n';
```

A karakterliterált **egyszeres idézőjelek** közé írjuk.

```
'A'  
'1'  
'\n'
```

Az üres karakter:

```
''
```

nem értelmezett.

A `char` alapértelmezett értéke:

```
'\0'
```

2. `string` — karakterek sorozata

A `string` karakterekből álló szöveget tárol.

```
string question = "Vincent! We happy?";
```

A `string` literált **dupla idézőjelek** közé írjuk.

Az üres `string` érvényes:

```
string empty = "";
```

Ez nem ugyanaz, mint:

```
string nothing = null;
```

```
"" → létező string, 0 karakterrel  
null → nincs string objektum
```

3. Konverziók

`ToString()`

Más típusú értéket stringgé alakít:

```
double pi = 3.14159265359;  
string text = pi.ToString();
```

Eredmény:

```
"3.14159265359"
```

További példák:

```
int number = 42;  
string a = number.ToString();  
  
bool value = true;  
string b = value.ToString();
```

`ToCharArray()`

Stringből karaktertömb készíthető:

```
string text = "Hello";  
char[] chars = text.ToCharArray();
```

A tömb tartalma:

```
'H', 'e', 'l', 'l', 'o'
```

4. Karaktervizsgálatok

A `char` típus több hasznos statikus metódust biztosít.

```
bool isLetter = char.IsLetter('a'); // true  
bool isDigit = char.IsDigit('1');  // true  
bool isUpper = char.IsUpper('a');  // false  
bool isLower = char.IsLower('a');  // true
```

Kis- és nagybetűs alak:

```
char lower = char.ToLower('A'); // 'a'  
char upper = char.ToUpper('a'); // 'A'
```

A metódusok visszatérési értéket adnak; az eredeti karaktert nem módosítják.

5. Karakterkódok

Egy `char` mögött numerikus karakterkód található.

```
int codeOfA = (int)'A';
```

Eredmény:

```
65
```

A fordított konverzió:

```
char letterZ = (char)90;
```

Eredmény:

```
'z'
```

A karakterkód miatt karaktereken a `++` és `--` operátor is értelmezett.

```
char c = 'A';  
c++;
```

Ez a következő karakterkódhoz tartozó karaktert eredményezi.

6. String indexelése

A string karakterei indexelő operátorral érhetőek el, ugyanúgy 0-alapúan, mint tömb esetén.

```
string quote = "Okay."  
  
char firstChar = quote[0];  
char lastChar = quote[quote.Length - 1];
```

Eredmény:

```
firstChar = 'O'  
lastChar = '.'
```

A `quote[i]` típusa `char`.

7. String hossza — Length

```
string text = "Hello";  
int length = text.Length;
```

Eredmény:

```
5
```

Az utolsó index:

```
text.Length - 1
```

Tipikus bejárás:

```
for (int i = 0; i < text.Length; i++)
{
    Console.WriteLine(text[i]);
}
```

Vagy foreach ciklussal:

```
foreach (char c in text)
{
    Console.WriteLine(c);
}
```

8. A string immutable

A string **immutable**, vagyis létrehozás után a tartalma nem módosítható helyben.

Ez hibás:

```
string text = "hello";
// text[0] = 'H';
```

A karaktert kiolvashatjuk:

```
char c = text[0];
```

De közvetlenül nem írhatjuk felül.

9. String módosítása karaktertömbbel

Ha elemenként szeretnénk módosítani a szöveget:

```
string quote = "easy peasy";

char[] quoteAsArray = quote.ToCharArray();
quoteAsArray[0] = 'E';

string modQuote = new string(quoteAsArray);
```

Eredmény:

```
"Easy peasy"
```

A folyamat:

```
string
  ↓ ToCharArray()
char[]
  ↓ módosítás
char[]
  ↓ new string(...)
string
```

10. String.Join()

A `Join` egy gyűjtemény elemeit egyetlen stringgé fűzi össze.

```
string[] words = { "x", "y", "z" };
string result = String.Join('-', words);
```

Eredmény:

```
"x-y-z"
```

Az első paraméter az elemek közé kerülő elválasztó.

11. IsNullOrEmpty() és IsNullOrWhiteSpace()

String.IsNullOrEmpty()

Igaz, ha a `string` `null` vagy üres.

```
string nothing = "";

bool isEmpty = String.IsNullOrEmpty(nothing);
```

Eredmény:

```
true
```

String.IsNullOrEmpty()

Igaz akkor is, ha a string csak whitespace karaktereket tartalmaz.

```
string something = " ";  
  
bool result = String.IsNullOrEmpty(something);
```

Eredmény:

```
true
```

Whitespace például:

```
space, tab, sortörés
```

12. Keresés stringben

Contains()

```
string quote = "EasyPeasy";  
bool contains = quote.Contains('P');
```

Eredmény:

```
true
```

IndexOf()

Az első előfordulás indexét adja vissza.

```
int firstIdx = quote.IndexOf('y');
```

Eredmény:

LastIndexOf()

Az utolsó előfordulás indexét adja vissza.

```
int lastIdx = quote.LastIndexOf("Pe");
```

Eredmény:

4

Ha nincs találat:

```
int noIdx = quote.IndexOf("OK");
```

Eredmény:

-1

Ugyanez igaz a `LastIndexOf()` esetén is.

13. Kis- és nagybetűs string

```
string quote = "Big Kahuna Burger";
```

```
string lower = quote.ToLower();
```

```
string upper = quote.ToUpper();
```

Eredmény:

```
"big kahuna burger"
```

```
"BIG KAHUNA BURGER"
```

Az eredeti `quote` nem változik.

14. Insert()

Szöveget szúr be egy adott indexre.

```
string quote = "Big Kahuna Burger";  
  
string inserted = quote.Insert(11, "Cheese ");
```

Eredmény:

```
"Big Kahuna Cheese Burger"
```

15. Remove()

Karaktereket távolít el egy megadott indextől.

```
string quote = "Big Kahuna Burger";  
  
string removed =  
    quote.Remove(quote.IndexOf("Kahuna"), 7);
```

Eredmény:

```
"Big Burger"
```

A második paraméter az eltávolítandó karakterek száma.

16. Replace()

Karaktert vagy részstringet cserél.

```
string quote = "Big Kahuna Burger";  
  
string replaced = quote.Replace('B', 'K');
```

Eredmény:

```
"Kig Kahuna Kurger"
```

Részstringgel is használható:

```
string result =  
    quote.Replace("Burger", "Cheeseburger");
```

17. Substring()

Egy string egy részét adja vissza.

```
string quote = "Big Kahuna Burger";  
  
string part = quote.Substring(4, 6);
```

Eredmény:

```
"Kahuna"
```

Általánosan:

```
Substring(kezdőIndex, hossz)
```

18. Split()

A `Split` egy stringet több részre bont, és egy `string[]` tömböt ad vissza.

```
string quote = "Big Kahuna Burger";  
  
string[] parts = quote.Split(' ');
```

Eredmény:

```
{ "Big", "Kahuna", "Burger" }
```

Az elválasztó nem kerül bele az eredmény elemeibe.

19. Trim()

A `Trim()` eltávolítja a string elejéről és végéről a whitespace karaktereket.

```
string input = "  hello  ";  
string cleaned = input.Trim();
```

Eredmény:

```
"hello"
```

A string belsejében lévő szóközöket nem törli.

20. String összehasonlítás

Lexikografikus összehasonlítás:

```
int result = String.Compare("Marsellus", "Jules");
```

Az eredmény előjele jelzi a sorrendet:

```
0    → azonos  
< 0 → első megelőzi a másodikat  
> 0 → első követi a másodikat
```

Egyszerű egyenlőségvizsgálat:

```
if (text1 == text2)  
{  
    // a két string azonos  
}
```

21. String interpoláció

A `$` segítségével változókat és kifejezéseket illeszthetünk stringbe.

```
string name = "Vincent";  
int age = 30;  
  
string text = $"{name} is {age} years old.";
```

Kifejezés is megadható:

```
int a = 5;
int b = 3;

Console.WriteLine($"{a} + {b} = {a + b}");
```

22. Formázás interpolációban

```
double value = 13.0;
string text = $"{value:F3}";
```

Eredmény:

```
13.000
```

Példa:

```
double normalTime = 13.0;
int mrWolfeTime = 10;

string expression =
    $"That's {normalTime:F3} minutes away. I'll be there in {mrWolfeTime}.";
```

23. Tipikus feldolgozási minták

Betűk és számjegyek számlálása

```
int letters = 0;
int digits = 0;

foreach (char c in text)
{
    if (char.IsLetter(c))
        letters++;

    if (char.IsDigit(c))
        digits++;
}
```

String visszafelé bejárása

```
for (int i = text.Length - 1; i >= 0; i--)
{
    Console.Write(text[i]);
}
```

Ez palindrom ellenőrzésnél is használható.

Tipikus indexpár:

```
text[i]
text[text.Length - 1 - i]
```

24. Karakterek szűrése

```
foreach (char c in text)
{
    if (char.IsLetterOrDigit(c))
    {
        // feldolgozás
    }
}
```

Ez jól használható például:

- rendszám normalizálásnál,
- e-mail ellenőrzésnél,
- palindrom előfeldolgozásnál,
- azonosítók generálásánál.

25. Split() és táblázatos adat

Példa bemenet:

```
Vincent;Vega;Vince\nMarsellus;Wallace;Big Man\nWinston;Wolf;The Wolf
```

Először sorokra bontjuk:

```
string[] rows = text.Split('\n');
```

Majd minden sort oszlopokra:

```
string[] cols = rows[i].Split(';');
```

Ezután az értékek kétdimenziós tömbbe helyezhetők.

A feldolgozási minta:

```
string  
    ↓ Split('\n')  
string[] sorok  
    ↓ soronként Split(';')  
string[] oszlopok  
    ↓  
kétdimenziós struktúra
```

26. Gyors összefoglaló

char

```
char c = 'A';
```

Alapérték:

```
'\0'
```

Vizsgálatok:

```
char.IsLetter(c)  
char.IsDigit(c)  
char.IsUpper(c)  
char.IsLower(c)
```

Konverzió:

```
(int)c  
(char)65
```

string

```
string text = "Hello";
```

Index:

```
text[0]
```

Hossz:

```
text.Length
```

String → karaktertömb:

```
text.ToArray()
```

Más típus → string:

```
value.ToString()
```

27. Fontos string metódusok

```
String.IsNullOrEmpty(...)  
String.IsNullOrWhiteSpace(...)
```

```
Contains(...)  
IndexOf(...)  
LastIndexOf(...)
```

```
ToLower()  
ToUpper()  
Trim()
```

```
Insert(...)  
Remove(...)  
Replace(...)  
Substring(...)  
Split(...)
```

```
String.Join(...)
```

28. Amit tudni kell a feladatok előtt

- `char` egyetlen karaktert tárol.
- `char` literálhoz egyszeres idézőjel kell.
- `string` karakterek sorozata.

- `""` érvényes üres string, és nem ugyanaz, mint `null` .
- A string indexelése 0-tól indul.
- `Length` a karakterek számát adja.
- A `string` immutable.
- Elemenkénti módosításhoz használható a `ToCharArray()` .
- `Contains()` előfordulást vizsgál.
- `IndexOf()` és `LastIndexOf()` indexet ad, sikertelen keresésnél `-1` -et.
- `Split()` egy `string[]` tömböt állít elő.
- `Join()` gyűjtemény elemeit fűzi össze.
- A legtöbb string-művelet **új stringet ad vissza**, az eredetit nem módosítja.

A fő technikai kérdés: a szöveget karakterek sorozataként hogyan járjuk be, vizsgáljuk, bontjuk fel vagy építjük újra?